

INTERNNOVA

WEEK 3 ASSIGNMENT

SQL & Excel for Data Analytics

Name: Prensi

Internship: Data Analytics Internship

Week: 3

Assignment: SQL & Excel for Data Analytics

1. Introduction

This report presents the work completed for Week 3 of the InternNova Data Analytics Internship.

The assignment focused on two important tools used in data analytics:

- **SQL** for working with databases and extracting useful information.
- **Microsoft Excel** for data formatting, filtering, analysis, lookup functions, Pivot Tables, and charts.

The work was divided into nine tasks covering basic SQL queries, filtering, aggregation, grouping, joins, subqueries, Excel functions, lookup functions, Pivot Tables, and charts.

2. Objectives

The main objectives of this assignment were:

1. Understand the basic concepts of databases.
 2. Create and work with tables using SQL.
 3. Retrieve specific data using SELECT queries.
 4. Filter and sort data using WHERE and ORDER BY.
 5. Use aggregate functions such as COUNT, SUM, AVG, MIN, and MAX.
 6. Understand GROUP BY and HAVING.
 7. Perform INNER JOIN, LEFT JOIN, and RIGHT JOIN.
 8. Understand and use SQL subqueries.
 9. Perform data formatting, sorting, and filtering in Excel.
 10. Use IF, COUNTIF, and SUMIF functions.
 11. Understand VLOOKUP and XLOOKUP.
 12. Create a Pivot Table and chart for data analysis.
-

3. Tools Used

The following tools were used to complete the assignment:

- **MySQL Workbench** – SQL database and queries
 - **Microsoft Excel** – Data analysis and visualization
 - **Google Drive** – File storage and sharing
-

TASK 1: Introduction to Databases & SELECT

4. Database Creation

A database named InternNova_Week3 was created in MySQL.

```
CREATE DATABASE InternNova_Week3;
```

```
USE InternNova_Week3;
```

```
SELECT DATABASE();
```

The database was successfully created and selected for the assignment.

4.1 Students Table

A Students table was created with the following columns:

- StudentID
- StudentName
- Age
- Course
- City
- Marks

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50),  
    Age INT,  
    Course VARCHAR(50),  
    City VARCHAR(50),  
    Marks DECIMAL(5,2)  
);
```

Ten sample student records were inserted into the table.

4.2 Display All Records

```
SELECT * FROM Students;
```

This query displays all records and columns from the Students table.

Screenshot: Task 1 – Select All Students

4.3 Select Specific Columns

```
SELECT StudentID, StudentName, Course, Marks
```

```
FROM Students;
```

This query displays only the required columns instead of the complete table.

Screenshot: Task 1 – Specific Columns

4.4 Column Aliases

```
SELECT
```

```
    StudentName AS Name,
```

```
    Course AS Program,
```

```
    Marks AS Score
```

```
FROM Students;
```

Column aliases are used to display more readable or user-friendly column names in the output.

Screenshot: Task 1 – Column Alias

Task 1 Result

The database and Students table were successfully created. Basic SELECT queries and column aliases were successfully executed.

TASK 2: WHERE, ORDER BY & Aggregate Functions

5. WHERE Clause

The WHERE clause was used to filter records based on specific conditions.

Students enrolled in BCA

```
SELECT *
```

```
FROM Students
```

```
WHERE Course = 'BCA';
```

This query displays only students whose course is BCA.

Students with marks greater than 85

```
SELECT StudentName, Course, Marks
```

```
FROM Students
```

```
WHERE Marks > 85;
```

This query returns students who scored more than 85 marks.

Screenshot: Task 2 – WHERE

5.1 ORDER BY

ORDER BY was used to arrange the results in ascending or descending order.

Descending Order

```
SELECT StudentName, Course, Marks
```

```
FROM Students
```

```
ORDER BY Marks DESC;
```

This displays students from highest marks to lowest marks.

Ascending Order

```
SELECT StudentName, Course, Marks
```

```
FROM Students
```

```
ORDER BY Marks ASC;
```

This displays students from lowest marks to highest marks.

5.2 Aggregate Functions

The following aggregate functions were used:

COUNT

```
SELECT COUNT(*) AS Total_Students  
FROM Students;
```

It returns the total number of students.

SUM

```
SELECT SUM(Marks) AS Total_Marks  
FROM Students;
```

It calculates the total marks.

AVG

```
SELECT AVG(Marks) AS Average_Marks  
FROM Students;
```

It calculates the average marks.

MIN

```
SELECT MIN(Marks) AS Minimum_Marks  
FROM Students;
```

It returns the minimum marks.

MAX

```
SELECT MAX(Marks) AS Maximum_Marks  
FROM Students;
```

It returns the maximum marks.

Combined Aggregate Query

```
SELECT  
    COUNT(*) AS Total_Students,  
    SUM(Marks) AS Total_Marks,
```

```
AVG(Marks) AS Average_Marks,  
MIN(Marks) AS Minimum_Marks,  
MAX(Marks) AS Maximum_Marks
```

```
FROM Students;
```

This query provides multiple summary statistics in a single result.

Task 2 Result

WHERE, ORDER BY, and aggregate functions were successfully used to filter, sort, and summarize student data.

TASK 3: GROUP BY & HAVING

6. GROUP BY

GROUP BY was used to organize records into groups and calculate summary values for each group.

Students by Course

```
SELECT Course, COUNT(*) AS Total_Students  
FROM Students  
GROUP BY Course;
```

This shows the number of students in each course.

Average Marks by Course

```
SELECT Course, AVG(Marks) AS Average_Marks  
FROM Students  
GROUP BY Course;
```

This calculates the average marks for each course.

Students by City

```
SELECT City, COUNT(*) AS Total_Students  
FROM Students  
GROUP BY City;
```

This shows the number of students from each city.

6.1 HAVING

HAVING was used to filter grouped results.

```
SELECT City, COUNT(*) AS Total_Students  
FROM Students  
GROUP BY City  
HAVING COUNT(*) >= 2;
```

This displays only those cities having at least two students.

Another example:

```
SELECT Course, AVG(Marks) AS Average_Marks  
FROM Students  
GROUP BY Course  
HAVING AVG(Marks) > 85;
```

This displays courses whose average marks are greater than 85.

Task 3 Result

GROUP BY and HAVING were successfully used to perform grouped analysis and filter aggregated results.

TASK 4: SQL JOINS

7. Departments Table

A second table named Departments was created.

```
CREATE TABLE Departments (  
    DepartmentID INT PRIMARY KEY,  
    Course VARCHAR(50),  
    DepartmentHead VARCHAR(50)  
);
```

Three department records were inserted.

The common column used for joining the tables was:

Course

7.1 INNER JOIN

```
SELECT  
    s.StudentID,  
    s.StudentName,  
    s.Course,  
    d.DepartmentHead  
FROM Students s  
INNER JOIN Departments d  
ON s.Course = d.Course;
```

Explanation

INNER JOIN returns only records where a matching Course exists in both tables.

7.2 LEFT JOIN

```
SELECT  
    s.StudentID,
```

```
s.StudentName,  
s.Course,  
d.DepartmentHead
```

```
FROM Students s
```

```
LEFT JOIN Departments d
```

```
ON s.Course = d.Course;
```

Explanation

LEFT JOIN returns all records from the Students table and matching records from the Departments table. If there is no match, NULL is displayed for the department information.

7.3 RIGHT JOIN

```
SELECT
```

```
s.StudentID,  
s.StudentName,  
d.Course,  
d.DepartmentHead
```

```
FROM Students s
```

```
RIGHT JOIN Departments d
```

```
ON s.Course = d.Course;
```

Explanation

RIGHT JOIN returns all records from the Departments table and matching records from the Students table.

The Departments table contains an MCA record that does not have a matching course in the Students table. Therefore, NULL values appear for the student columns for that unmatched department.

7.4 Sorted JOIN Result

```
SELECT
```

```
s.StudentName,  
s.Course,  
d.DepartmentHead  
FROM Students s  
INNER JOIN Departments d  
ON s.Course = d.Course  
ORDER BY s.Course, s.StudentName;
```

Task 4 Result

INNER JOIN, LEFT JOIN, and RIGHT JOIN were successfully executed and their differences were observed through the output.

TASK 5: SQL SUBQUERIES

8. Students Above Average Marks

A subquery was used to find students whose marks were higher than the overall average.

```
SELECT StudentName, Course, Marks  
FROM Students  
WHERE Marks > (  
    SELECT AVG(Marks)  
    FROM Students  
);
```

Explanation

The inner query calculates the average marks. The outer query then finds students whose marks are greater than that average.

8.1 Student with Highest Marks

```
SELECT StudentName, Course, Marks  
FROM Students
```

```
WHERE Marks = (  
    SELECT MAX(Marks)  
    FROM Students  
);
```

Explanation

The subquery finds the maximum marks, and the outer query returns the student having those marks.

8.2 Course with Highest Average Marks

```
SELECT StudentName, Course, Marks  
FROM Students  
WHERE Course = (  
    SELECT Course  
    FROM Students  
    GROUP BY Course  
    ORDER BY AVG(Marks) DESC  
    LIMIT 1  
);
```

Explanation

The inner query calculates the average marks for each course and identifies the course with the highest average. The outer query displays students from that course.

Task 5 Result

SQL subqueries were successfully used to solve analytical problems involving average, maximum, and grouped values.

TASK 6: Excel Formatting, Sorting & Filtering

9. Excel Dataset

A Sales dataset was created in Microsoft Excel with the following columns:

- OrderID
- Product
- Category
- Region
- Sales
- Quantity
- Profit

The dataset contained 15 records.

9.1 Formatting

The following formatting was applied:

- Header row was made bold.
- Headers were aligned properly.
- Header cells were filled and bordered.
- Column widths were adjusted.
- Sales and Profit columns were formatted as numbers/currency.
- The data range was converted into an Excel Table.

9.2 Sorting

The Sales column was sorted from largest to smallest.

This made it easier to identify products with higher sales.

9.3 Filtering

Filters were applied to the dataset.

Region Filter

The Region column was filtered for **North**.

Category Filter

The Category column was filtered for **Technology**.

Task 6 Result

Excel formatting, sorting, and filtering were successfully completed.

Screenshots:

- Task 6 – Sorted Sales
- Task 6 – North Region Filter
- Task 6 – Technology Category Filter

TASK 7: Conditional Formatting & Excel Functions

10. Conditional Formatting

Conditional formatting was applied to the Profit column to highlight values greater than 1000.

This makes higher-profit records easier to identify visually.

10.1 IF Function

A new column named Profit_Status was created.

Formula:

=IF(G2>1000,"High Profit","Low Profit")

The formula was applied to all records.

Explanation

If Profit is greater than 1000, the result is **High Profit**. Otherwise, it is **Low Profit**.

10.2 COUNTIF Function

The number of Technology products was calculated using:

=COUNTIF(C2:C16,"Technology")

This counts the records where Category is Technology.

10.3 SUMIF Function

Total sales for Furniture products were calculated using:

=SUMIF(C2:C16,"Furniture",E2:E16)

This adds the Sales values where the Category is Furniture.

10.4 Summary Functions

A small summary section was created in Excel containing:

Metric	Function
Technology Products	COUNTIF
Furniture Sales	SUMIF
High Profit Products	COUNTIF

Task 7 Result

Conditional formatting and the IF, COUNTIF, and SUMIF functions were successfully applied for data analysis.

TASK 8: VLOOKUP & XLOOKUP

11. Lookup Data

A separate worksheet named Lookup_Data was created.

It contained:

- Product
- Category
- Region

This worksheet was used for lookup operations.

11.1 VLOOKUP – Category

Formula:

=VLOOKUP(E2,A2:C16,2,FALSE)

The searched product was **Laptop**.

The formula returned:

Technology

Explanation

VLOOKUP searches for a value in the first column of a table and returns a corresponding value from another column.

11.2 VLOOKUP – Region

Formula:

=VLOOKUP(E2,A2:C16,3,FALSE)

For Laptop, the returned region was:

North

11.3 XLOOKUP – Category

Formula:

=XLOOKUP(I2,A2:A16,B2:B16,"Not Found")

The searched product was **Smartphone**.

The returned category was:

Technology

11.4 XLOOKUP – Region

Formula:

=XLOOKUP(I2,A2:A16,C2:C16,"Not Found")

The returned region was:

East

Task 8 Result

Both VLOOKUP and XLOOKUP were successfully used to retrieve related information from another table.

TASK 9: PIVOT TABLE & CHART

12. Pivot Table

A Pivot Table was created from the Sales dataset.

The Pivot Table used:

- **Category** → Rows
- **Sales** → Values
- **Profit** → Values

This allowed category-wise sales and profit to be summarized quickly.

12.1 Pivot Chart

A column chart was created from the Pivot Table.

Chart title:

Category-wise Sales Analysis

Explanation

The Pivot Chart provides a visual comparison of sales across different product categories. It makes it easier to understand which categories contribute more to total sales.

Task 9 Result

A Pivot Table and Pivot Chart were successfully created for category-wise analysis.

13. Overall Learning

During this assignment, I learned how SQL and Excel can be used together for data analytics.

SQL Learning

I learned:

- Database creation
- Table creation
- SELECT statements
- WHERE conditions
- ORDER BY
- Aggregate functions
- GROUP BY
- HAVING
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- Subqueries

Excel Learning

I learned:

- Data formatting
- Sorting
- Filtering
- Conditional formatting
- IF function
- COUNTIF
- SUMIF
- VLOOKUP
- XLOOKUP

- Pivot Tables
- Pivot Charts

These skills are useful for cleaning, analyzing, summarizing, and presenting business data.

14. Conclusion

The InternNova Week 3 assignment helped me gain practical experience in SQL and Excel for data analytics.

Using SQL, I learned how to retrieve, filter, group, join, and analyze data stored in databases. I also learned how subqueries can be used to solve more advanced analytical problems.

Using Excel, I practiced data formatting, sorting, filtering, conditional formatting, functions, lookup operations, Pivot Tables, and charts.

Overall, this assignment improved my understanding of basic data analysis workflows and strengthened my practical skills in SQL and Excel.